

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO BÀI TẬP LỚN

MÔN HỌC: LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG

Đề tài:

XÂY DỰNG GAME DIAMOND RUSH 2D

SỬ DỤNG NGÔN NGỮ JAVA

Giảng viên hướng dẫn: TS. Trần Nhật Hóa

Nhóm sinh viên thực hiện: Nhóm 10

1. Nguyễn Huy Hoàng (Trưởng nhóm) - 202416921
2. Trương Đức Hoàng - 202416918
3. Nguyễn Hoàng Thái - 202417029
4. Nguyễn Anh Tuấn - 202417062
5. Nguyễn Hữu Vinh - 202417077

HÀ NỘI, 2026

Mục lục

1	THÔNG TIN CHUNG VỀ ĐỀ TÀI	3
1.1	Tên đề tài	3
1.2	Danh sách thành viên	3
2	PHÂN CÔNG CÔNG VIỆC VÀ ĐÁNH GIÁ	4
2.1	Bảng phân công công việc	4
3	TÓM TẮT KẾT QUẢ ĐẠT ĐƯỢC	5
3.1	Tổng quan kết quả sản phẩm	5
3.2	Các chức năng chính đã hoàn thành	5
3.2.1	Giao diện menu và điều hướng	5
3.2.2	Gameplay chính	6
3.2.3	Hệ thống bản đồ dạng CSV	7
3.2.4	Điều khiển nhân vật	7
3.2.5	Thu thập kim cương	7
3.2.6	Hệ thống đá và xử lý vật cản	8
3.2.7	Vật phẩm búa và tương tác phá vật cản	8
3.2.8	Enemy và bẫy trong game	9
3.2.9	Điều kiện thắng và thua	9
3.2.10	Lưu và tiếp tục tiến trình chơi	10
3.3	Kết quả về tổ chức mã nguồn	10
3.4	Liên hệ và vận dụng nội dung môn học OOP	11
3.4.1	Vận dụng tính đóng gói	11
3.4.2	Vận dụng tính kế thừa	11
3.4.3	Vận dụng tính đa hình	12
3.4.4	Vận dụng tính trừu tượng	12
3.5	Sơ đồ use case và sơ đồ lớp	13

3.5.1	Sơ đồ use case tổng quát	13
3.5.2	Sơ đồ lớp nhóm lõi game, thực thể và giao diện	14
3.5.3	Sơ đồ lớp nhóm object và vật phẩm	15
3.5.4	Nhận xét về thiết kế sơ đồ lớp	16
3.6	Đánh giá chung kết quả đạt được	16
4	KẾT LUẬN	18
4.1	Hướng phát triển cho bài tập lớn	18
4.2	Những điều học được từ bài tập lớn và môn học	18
4.3	Kết luận chung	19
	PHỤ LỤC	20
1.	Hướng dẫn chạy chương trình	20
1.1.	Chạy bằng file JAR	20
1.2.	Chạy trực tiếp từ mã nguồn	20
2.	Cấu trúc thư mục chính của project	21
3.	Link repository	22

CHƯƠNG 1. THÔNG TIN CHUNG VỀ ĐỀ TÀI

1.1 Tên đề tài

Tên đề tài của nhóm là:

XÂY DỰNG GAME DIAMOND RUSH 2D SỬ DỤNG NGÔN NGỮ JAVA

Diamond Rush 2D là game phiêu lưu – giải đố theo dạng ô lưới. Người chơi điều khiển nhân vật di chuyển trong bản đồ, thu thập kim cương, đẩy đá, sử dụng vật phẩm hỗ trợ, tránh kẻ địch và hoàn thành màn chơi. Dự án được xây dựng bằng Java, sử dụng Swing/AWT và Java2D để tạo giao diện, xử lý input và vẽ đồ họa.

1.2 Danh sách thành viên

Bảng 1.1: Danh sách thành viên nhóm 10

STT	Họ và tên	Mã sinh viên	Vai trò
1	Nguyễn Huy Hoàng	202416921	Trưởng nhóm
2	Trương Đức Hoàng	202416918	Thành viên
3	Nguyễn Hoàng Thái	202417029	Thành viên
4	Nguyễn Anh Tuấn	202417062	Thành viên
5	Nguyễn Hữu Vinh	202417077	Thành viên

CHƯƠNG 2. PHÂN CÔNG CÔNG VIỆC VÀ ĐÁNH GIÁ

2.1 Bảng phân công công việc

Bảng dưới đây trình bày phần phân công công việc, tỉ lệ đóng góp và điểm đánh giá nội bộ của từng thành viên theo thang điểm 10. Tỉ lệ đóng góp được đánh giá dựa trên khối lượng công việc thực tế trong quá trình xây dựng sản phẩm, hoàn thiện mã nguồn và chuẩn bị báo cáo.

Bảng 2.1: Bảng phân công công việc và đánh giá thành viên

Thành viên	Vai trò	Công việc thực hiện	Tỉ lệ	Điểm
Nguyễn Huy Hoàng	Trưởng nhóm / Lập trình chính	Phụ trách chính việc xây dựng mã nguồn game; thiết kế cấu trúc project; cài đặt gameplay chính; xử lý bản đồ, object, va chạm, nhân vật, vật phẩm, enemy, logic thắng thua; kiểm thử, chỉnh sửa và hoàn thiện báo cáo.	60%	10/10
Nguyễn Anh Tuấn	Hỗ trợ lập trình	Hỗ trợ xây dựng phần xử lý các nút điều khiển, thao tác di chuyển nhân vật và kiểm tra một số chức năng trong quá trình chơi.	15%	8/10
Nguyễn Hữu Vinh	Hỗ trợ giao diện	Hỗ trợ xây dựng và chỉnh sửa giao diện menu, các màn hình điều hướng cơ bản và một số thành phần hiển thị trong game.	15%	8/10
Trương Đức Hoàng	Hỗ trợ nhóm	Tham gia trao đổi ý tưởng, hỗ trợ kiểm tra sản phẩm, đóng góp nhận xét trong quá trình hoàn thiện game và báo cáo.	5%	6/10
Nguyễn Hoàng Thái	Hỗ trợ nhóm	Tham gia góp ý gameplay, hỗ trợ rà soát lỗi và kiểm tra sản phẩm cuối trước khi nộp.	5%	6/10

CHƯƠNG 3. TÓM TẮT KẾT QUẢ ĐẠT ĐƯỢC

3.1 Tổng quan kết quả sản phẩm

Sau quá trình thực hiện bài tập lớn, nhóm đã xây dựng được một game Diamond Rush 2D bằng ngôn ngữ Java. Sản phẩm có thể chạy trên máy tính dưới dạng ứng dụng desktop, có giao diện đồ họa, có màn hình menu, màn hình gameplay, các vật phẩm tương tác, enemy, điều kiện thắng thua và cơ chế lưu tiến trình chơi.

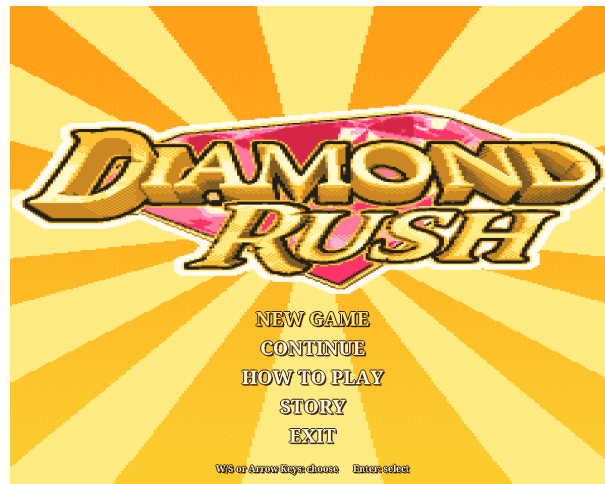
Game được phát triển theo phong cách phiêu lưu – giải đố dạng ô lưới. Người chơi điều khiển nhân vật chính di chuyển trong bản đồ, thu thập kim cương, đẩy đá, sử dụng búa, tránh enemy và vượt qua các chướng ngại vật để hoàn thành màn chơi. Dữ liệu màn chơi được tách khỏi mã nguồn thông qua các file CSV, giúp việc thiết kế và chỉnh sửa map trở nên thuận tiện hơn.

Về mặt kỹ thuật, dự án không chỉ dừng ở việc hiển thị hình ảnh mà còn cài đặt nhiều thành phần quan trọng của một game 2D như game loop, xử lý input, vẽ đồ họa bằng Java2D, đọc dữ liệu bản đồ, kiểm tra va chạm, quản lý trạng thái game, xử lý vật phẩm, enemy, âm thanh và giao diện người dùng.

3.2 Các chức năng chính đã hoàn thành

3.2.1 Giao diện menu và điều hướng

Game có giao diện menu chính để người chơi lựa chọn các chức năng cơ bản như bắt đầu game, tiếp tục tiến trình, xem hướng dẫn, xem cốt truyện hoặc thoát game. Menu giúp người chơi tiếp cận game dễ dàng hơn thay vì vào thẳng màn chơi.



Hình 3.1: Giao diện menu chính của game

Giao diện menu là một phần quan trọng trong trải nghiệm người dùng. Việc tách trạng thái menu khỏi trạng thái gameplay giúp chương trình dễ quản lý hơn, đồng thời thể hiện cách tổ chức các màn hình bằng hướng đối tượng.

3.2.2 Gameplay chính

Màn hình gameplay là nơi hiển thị bản đồ, nhân vật, vật phẩm, enemy và các chướng ngại vật. Người chơi điều khiển nhân vật bằng bàn phím, di chuyển theo từng ô để giải đố và thu thập kim cương.



Hình 3.2: Giao diện gameplay chính

Trong quá trình chơi, hệ thống liên tục cập nhật trạng thái của người chơi, object, enemy và bản đồ. Sau mỗi frame, game vẽ lại màn hình để phản ánh trạng thái mới. Đây là cơ chế cơ bản của game loop trong các trò chơi 2D.

3.2.3 Hệ thống bản đồ dạng CSV

Một kết quả quan trọng của dự án là bản đồ được thiết kế dưới dạng file CSV thay vì viết trực tiếp trong mã nguồn. Mỗi màn chơi gồm dữ liệu nền và dữ liệu object. Cách tổ chức này giúp nhóm dễ dàng chỉnh sửa, mở rộng hoặc thêm màn chơi mới mà không cần thay đổi nhiều logic Java.

Các tile nền có thể đại diện cho đất, tường, bụi cây, nhựa, điểm spawn hoặc khóa. Các object có thể đại diện cho đá, kim cương, cửa, búa, chìa khóa, enemy, công tắc, tượng phun lửa và các vật cản đặc biệt.

Việc đọc map từ CSV được xử lý bởi lớp MapLoader. Lớp này có nhiệm vụ đọc dữ liệu từ file, lưu thành ma trận và hỗ trợ các phương thức kiểm tra tile như ô nào có thể đi qua, ô nào là tường, ô nào là vật cản. Đây là một phần giúp tách dữ liệu màn chơi khỏi phần xử lý logic.

3.2.4 Điều khiển nhân vật

Nhân vật người chơi được cài đặt trong lớp Player. Người chơi có thể di chuyển lên, xuống, trái, phải bằng bàn phím. Mỗi lần di chuyển, game kiểm tra ô đích trước khi cập nhật vị trí nhân vật.

Nếu ô đích là ô trống hoặc có vật phẩm có thể nhặt, nhân vật được phép di chuyển. Nếu ô đích là tường, khóa hoặc vật cản không thể đi qua, nhân vật bị chặn lại. Nếu ô đích là đá, game kiểm tra xem đá có thể bị đẩy sang ô tiếp theo hay không. Nếu hợp lệ, đá được đẩy đi và nhân vật tiến vào vị trí cũ của đá.

Cách xử lý này giúp gameplay có tính giải đố rõ ràng, vì người chơi phải tính toán hướng đi, vị trí đá và đường tới kim cương hoặc cửa thoát.

3.2.5 Thu thập kim cương

Kim cương là vật phẩm quan trọng nhất trong game. Khi người chơi di chuyển vào ô chứa kim cương, kim cương biến mất khỏi bản đồ và điểm số của người chơi tăng lên.



Hình 3.3: Nhân vật thu thập kim cương trong màn chơi

Chức năng nhặt kim cương được cài đặt thông qua tương tác giữa Player và các object kế thừa từ GameObject. Khi nhân vật bước vào ô có object, chương trình kiểm tra loại object và thực hiện hành động tương ứng. Với kim cương, hành động là tăng điểm, tắt trạng thái hoạt động của object và phát hiệu ứng âm thanh nếu có.

3.2.6 Hệ thống đá và xử lý vật cản

Đá là một object có va chạm. Người chơi không thể đi xuyên qua đá, nhưng có thể đẩy đá nếu ô phía sau đá hợp lệ. Ngoài ra, đá có thể rơi xuống hoặc trượt trong một số điều kiện nhất định. Cơ chế này tạo thêm yếu tố giải đố cho game.

Việc xử lý đá yêu cầu kiểm tra nhiều điều kiện: vị trí hiện tại của đá, ô phía sau đá, tile nền, object khác, enemy và trạng thái rơi. Đây là một trong những phần thể hiện rõ tư duy xử lý logic game theo từng đối tượng.

3.2.7 Vật phẩm búa và tương tác phá vật cản

Búa là vật phẩm hỗ trợ giúp người chơi tương tác với môi trường. Sau khi nhặt được búa, người chơi có thể sử dụng búa để phá vật cản hoặc tác động lên enemy ở ô kề bên.



Hình 3.4: Nhân vật nhặt được vật phẩm búa

Khi người chơi dùng búa, game xác định hướng nhân vật đang quay mặt và kiểm tra ô kề bên theo hướng đó. Nếu ô này chứa vật cản có thể phá, game sẽ xử lý phá vật cản. Nếu ô này chứa enemy, búa có thể làm choáng enemy tùy theo logic đã cài đặt.



Hình 3.5: Nhân vật dùng búa để phá vật cản

Chức năng dùng búa giúp gameplay đa dạng hơn. Người chơi không chỉ di chuyển và nhặt vật phẩm mà còn có thể chủ động tác động đến môi trường xung quanh để mở đường hoặc xử lý nguy hiểm.

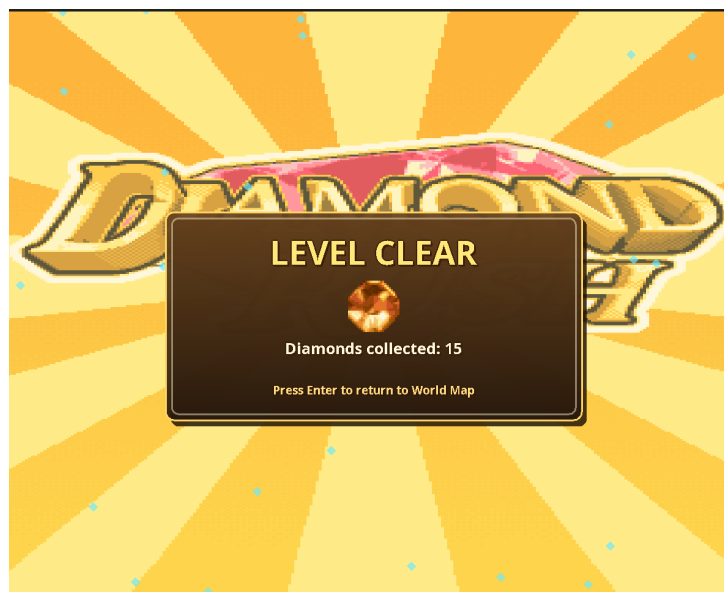
3.2.8 Enemy và bẫy trong game

Game có enemy dạng rắn và các bẫy như tượng phun lửa. Enemy có thể di chuyển qua lại theo một trục và đổi hướng khi gặp vật cản. Nếu người chơi chạm enemy hoặc vùng nguy hiểm, người chơi bị mất mạng.

Tượng phun lửa hoạt động theo chu kỳ, tạo vùng nguy hiểm trên bản đồ. Người chơi cần quan sát thời điểm lửa xuất hiện để tránh bị mất mạng. Các yếu tố này giúp game có độ thử thách cao hơn, không chỉ là thu thập kim cương đơn thuần.

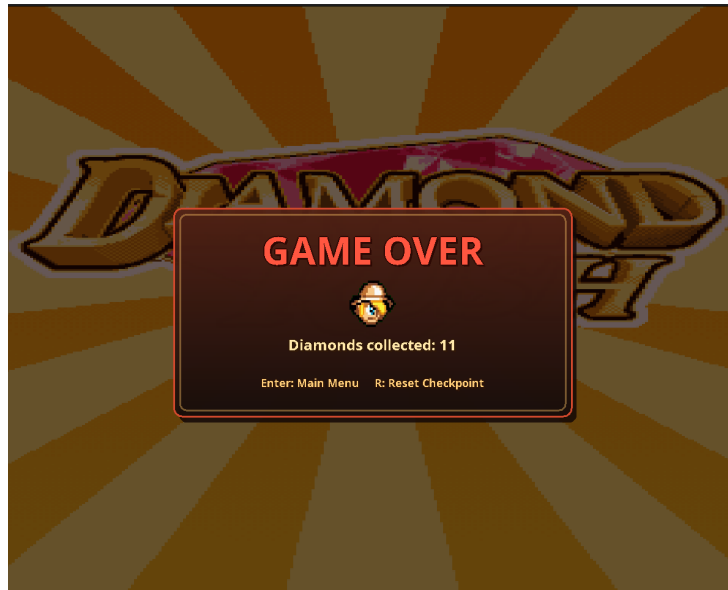
3.2.9 Điều kiện thắng và thua

Game có trạng thái thắng và thua rõ ràng. Người chơi thắng khi hoàn thành màn chơi hoặc hoàn thành màn cuối. Người chơi thua khi mất hết mạng do chạm enemy, bẫy hoặc các nguy hiểm khác.



Hình 3.6: Màn hình chiến thắng

Màn hình chiến thắng giúp thông báo cho người chơi rằng đã hoàn thành mục tiêu của game hoặc màn chơi cuối cùng.



Hình 3.7: Màn hình thua cuộc

Màn hình thua cuộc xuất hiện khi người chơi không còn mạng. Việc tách màn hình kết thúc thành trạng thái riêng giúp game có luồng hoạt động rõ ràng hơn.

3.2.10 Lưu và tiếp tục tiến trình chơi

Dự án có cơ chế lưu tiến trình chơi để người chơi có thể tiếp tục ở lần chạy sau. Đây là chức năng giúp game hoàn chỉnh hơn so với một bản demo đơn giản. Thông tin lưu có thể bao gồm màn chơi hiện tại, trạng thái mở khóa màn, số kim cương hoặc một số trạng thái cần thiết khác.

Cơ chế save/load cũng giúp nhóm hiểu rõ hơn cách lưu dữ liệu ngoài chương trình và khôi phục trạng thái game khi chạy lại.

3.3 Kết quả về tổ chức mã nguồn

Mã nguồn của dự án được chia thành nhiều package theo chức năng, giúp việc quản lý chương trình rõ ràng hơn. Cấu trúc này giúp nhóm dễ tìm kiếm, sửa lỗi và mở rộng code.

Các nhóm chính trong mã nguồn gồm:

- `core`: chứa các lớp lõi như `GamePanel`, `GameLoop`.
- `entity`: chứa các thực thể động như `Player`, `Enemy`.
- `object`: chứa các object trong map như `Diamond`, `Rock`, `Door`, `Hammer`, `Key`.
- `map`: chứa lớp đọc và xử lý bản đồ.
- `input`: chứa lớp xử lý bàn phím.
- `ui`: chứa các màn hình giao diện và trạng thái game.

- `util`: chứa các lớp tiện ích như quản lý tài nguyên.

Việc chia package như trên giúp dự án có cấu trúc tương đối rõ ràng. Mỗi nhóm class phụ trách một phần riêng của game, phù hợp với tư duy lập trình hướng đối tượng.

3.4 Liên hệ và vận dụng nội dung môn học OOP

Một trong những mục tiêu quan trọng của bài tập lớn là vận dụng kiến thức môn Lập trình hướng đối tượng vào sản phẩm thực tế. Trong dự án Diamond Rush 2D, nhóm đã áp dụng các nội dung OOP ở nhiều phần của chương trình.

3.4.1 Vận dụng tính đóng gói

Tính đóng gói được thể hiện qua việc mỗi lớp quản lý một nhóm dữ liệu và hành vi riêng. Ví dụ, lớp `Player` quản lý trạng thái và hành vi của nhân vật người chơi, lớp `Enemy` quản lý enemy, lớp `MapLoader` quản lý dữ liệu bản đồ, còn lớp `GameObject` lưu trữ các thông tin chung của object trong map.

Nhờ đóng gói, các chức năng của game được tách thành các thành phần nhỏ hơn thay vì viết toàn bộ trong một file duy nhất. Điều này giúp chương trình dễ đọc, dễ sửa và dễ kiểm tra hơn.

Tuy nhiên, dự án vẫn còn một số điểm có thể cải thiện. Một vài thuộc tính trong các class vẫn đang để ở mức truy cập rộng như `public` hoặc `protected`. Trong tương lai, nhóm có thể chuyển các thuộc tính này sang `private` và bổ sung getter/setter để tăng tính đóng gói.

3.4.2 Vận dụng tính kế thừa

Dự án sử dụng kế thừa để biểu diễn quan hệ giữa các đối tượng có đặc điểm chung. Ví dụ, `Player` và `Enemy` đều là các thực thể có vị trí, hướng di chuyển, vùng va chạm và khả năng vẽ lên màn hình, nên có thể kế thừa từ lớp cha `Entity`.

Tương tự, các object như `Diamond`, `Rock`, `Door`, `Hammer`, `Key`, `Statue`, `LockBolt` đều có những thuộc tính chung như vị trí, ảnh hiển thị, trạng thái hoạt động và trạng thái va chạm. Vì vậy, các lớp này kế thừa từ lớp cha `GameObject`.

Một số quan hệ kế thừa cụ thể trong dự án:

Bảng 3.1: Một số quan hệ kế thừa trong dự án

Lớp con	Lớp cha	Ý nghĩa
Player	Entity	Player là một thực thể có khả năng cập nhật và vẽ.
Enemy	Entity	Enemy là một thực thể có trạng thái và hành vi riêng.
Diamond	GameObject	Kim cương là một object trên bản đồ.
Rock	GameObject	Đá là một object có va chạm và có thể tương tác.
Door	GameObject	Cửa là object dùng để hoàn thành màn chơi.
SpecialLock	LockBolt	Khóa đặc biệt mở rộng hành vi từ khóa chốt.

Việc sử dụng kế thừa giúp giảm lặp mã nguồn và giúp hệ thống dễ mở rộng khi cần thêm object mới.

3.4.3 Vận dụng tính đa hình

Tính đa hình được thể hiện khi chương trình xử lý nhiều object khác nhau thông qua cùng một kiểu dữ liệu cha. Trong game, danh sách object có thể chứa nhiều đối tượng cụ thể như kim cương, đá, cửa, búa, chìa khóa, khóa, tượng phun lửa. Các đối tượng này đều có thể được quản lý dưới kiểu `GameObject`.

Khi vẽ object, chương trình có thể gọi chung phương thức `draw()` cho từng object. Tùy vào class thực thể của object, cách vẽ sẽ khác nhau. Ví dụ, `Diamond` vẽ hình kim cương, `Rock` vẽ hình đá, `Door` vẽ hình cửa.

Tương tự, `Player` và `Enemy` đều kế thừa từ `Entity` nhưng có cách triển khai `update()` và `draw()` khác nhau. Điều này cho thấy cùng một hành vi tổng quát nhưng mỗi lớp con có cách thực hiện riêng.

3.4.4 Vận dụng tính trừu tượng

Tính trừu tượng được thể hiện qua các lớp tổng quát như `Entity`, `GameObject` và interface `Screen`. Các lớp này không đại diện cho một đối tượng cụ thể trong game mà đại diện cho khái niệm chung.

`Entity` đại diện cho các thực thể có thể cập nhật và vẽ. `GameObject` đại diện cho các object xuất hiện trên bản đồ. `Screen` đại diện cho các màn hình giao diện có khả năng cập nhật và vẽ.

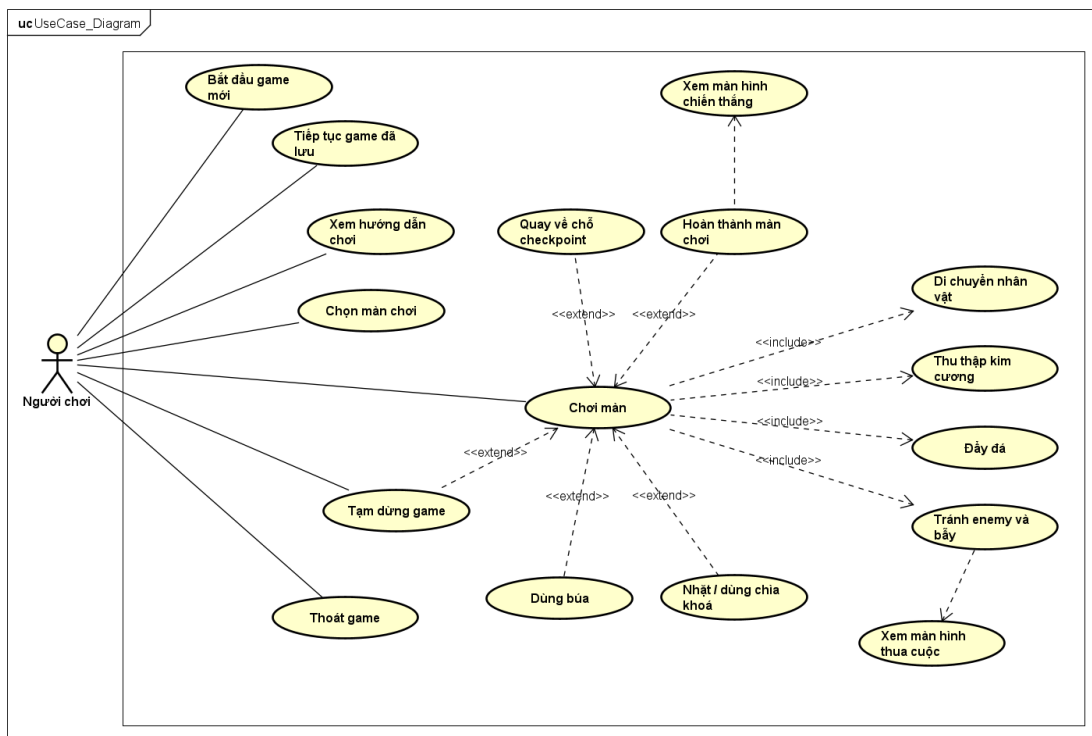
Nhờ trừu tượng, chương trình có thể định nghĩa trước các hành vi chung, sau đó các lớp con triển khai chi tiết theo nhu cầu của từng loại đối tượng.

3.5 Sơ đồ use case và sơ đồ lớp

Để mô tả rõ hơn chức năng và cấu trúc thiết kế của game Diamond Rush 2D, nhóm xây dựng ba sơ đồ chính: sơ đồ use case tổng quát, sơ đồ lớp nhóm lỗi game – thực thể – giao diện và sơ đồ lớp nhóm object – vật phẩm. Các sơ đồ này giúp thể hiện mối liên hệ giữa yêu cầu chức năng của game với cách tổ chức mã nguồn theo hướng đối tượng.

3.5.1 Sơ đồ use case tổng quát

Sơ đồ use case mô tả các chức năng chính mà người chơi có thể thực hiện trong hệ thống. Tác nhân chính của game là **Người chơi**. Người chơi có thể bắt đầu game mới, tiếp tục game đã lưu, xem hướng dẫn chơi, chọn màn chơi, tạm dừng game hoặc thoát game. Trong quá trình chơi màn, người chơi thực hiện các hành động như di chuyển nhân vật, thu thập kim cương, đẩy đá, tránh enemy và bẫy, dùng búa, nhặt / dùng chìa khóa, Quay về checkpoint, Hoàn thành màn chơi, Xem màn hình chiến thắng, Xem màn hình thua cuộc.



Hình 3.8: Sơ đồ use case tổng quát của game Diamond Rush 2D

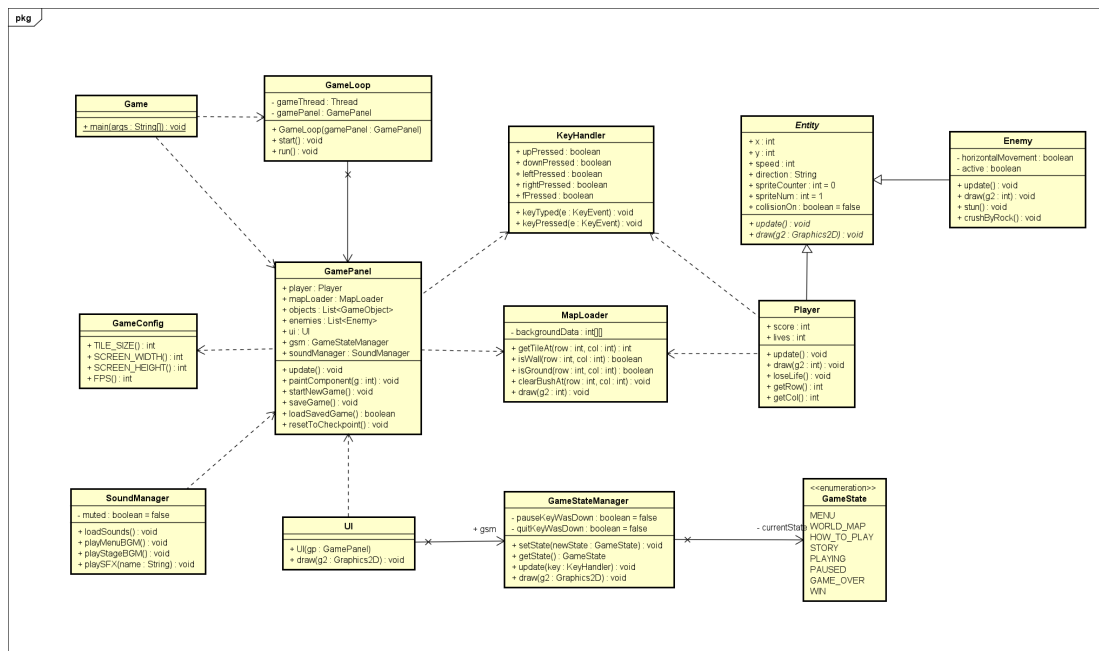
Trong sơ đồ, use case trung tâm là *Chơi màn*. Đây là chức năng chính của game, bao gồm nhiều hành động nhỏ hơn. Các chức năng như *Di chuyển nhân vật*, *Thu thập kim cương*, *Đẩy đá* và *Tránh enemy và bẫy* được biểu diễn bằng quan hệ *include* vì chúng là các hành động thường xuyên xảy ra trong quá trình chơi.

Một số chức năng khác như *Dùng búa*, *Nhặt / dùng chìa khóa*, *Quay về checkpoint*, *Tạm dừng game* và *Hoàn thành màn chơi* được biểu diễn bằng quan hệ *extend*. Đây là các chức năng mở rộng, chỉ xảy ra khi có điều kiện cụ thể. Ví dụ, người chơi chỉ có thể dùng búa sau khi đã nhặt được búa, chỉ có thể dùng chìa khóa khi đã có chìa khóa, hoặc chỉ hoàn thành màn khi đạt đủ điều kiện qua màn.

Sơ đồ cũng thể hiện hai luồng kết thúc quan trọng của game. Khi người chơi hoàn thành màn chơi, hệ thống chuyển tới màn hình chiến thắng. Ngược lại, khi người chơi không vượt qua được các nguy hiểm như enemy, bẫy hoặc mất hết mạng, hệ thống chuyển tới màn hình thua cuộc. Nhờ vậy, sơ đồ use case thể hiện được đầy đủ vòng đời cơ bản của một lượt chơi.

3.5.2 Sơ đồ lớp nhóm lõi game, thực thể và giao diện

Sơ đồ lớp thứ nhất mô tả các lớp đóng vai trò trung tâm trong quá trình vận hành game. Nhóm này bao gồm các lớp lõi như `Game`, `GameLoop`, `GamePanel`, `GameConfig`; các lớp thực thể như `Entity`, `Player`, `Enemy`; các lớp xử lý dữ liệu và điều khiển như `MapLoader`, `KeyHandler`; cùng các lớp giao diện và trạng thái như `UI`, `GameStateManager`, `GameState`, `SoundManager`.



Hình 3.9: Sơ đồ lớp nhóm lõi game, thực thể và giao diện

Trong sơ đồ này, Game là lớp khởi động chương trình. Lớp này tạo cửa sổ game, khởi tạo GamePanel và bắt đầu vòng lặp thông qua GameLoop. GameLoop chịu trách nhiệm gọi cập nhật logic và vẽ lại màn hình theo chu kỳ, giúp game hoạt động liên tục theo thời gian thực.

GamePanel là lớp trung tâm của hệ thống. Lớp này quản lý các thành phần quan trọng như Player, MapLoader, danh sách objects, danh sách enemies, UI, GameStateManager và SoundManager. Có thể xem GamePanel là nơi điều phối chính giữa gameplay, bản đồ, thực thể, vật phẩm, giao diện và âm thanh.

Lớp Entity được thiết kế là lớp cha cho các thực thể có khả năng cập nhật và vẽ lên màn hình. Hai lớp Player và Enemy kế thừa từ Entity. Đây là một ví dụ thể hiện tính kế thừa trong lập trình hướng đối tượng. Cả người chơi và enemy đều có các thuộc tính chung như tọa độ, tốc độ, hướng di chuyển và vùng va chạm, nhưng mỗi lớp lại có cách xử lý riêng trong phương thức update() và draw().

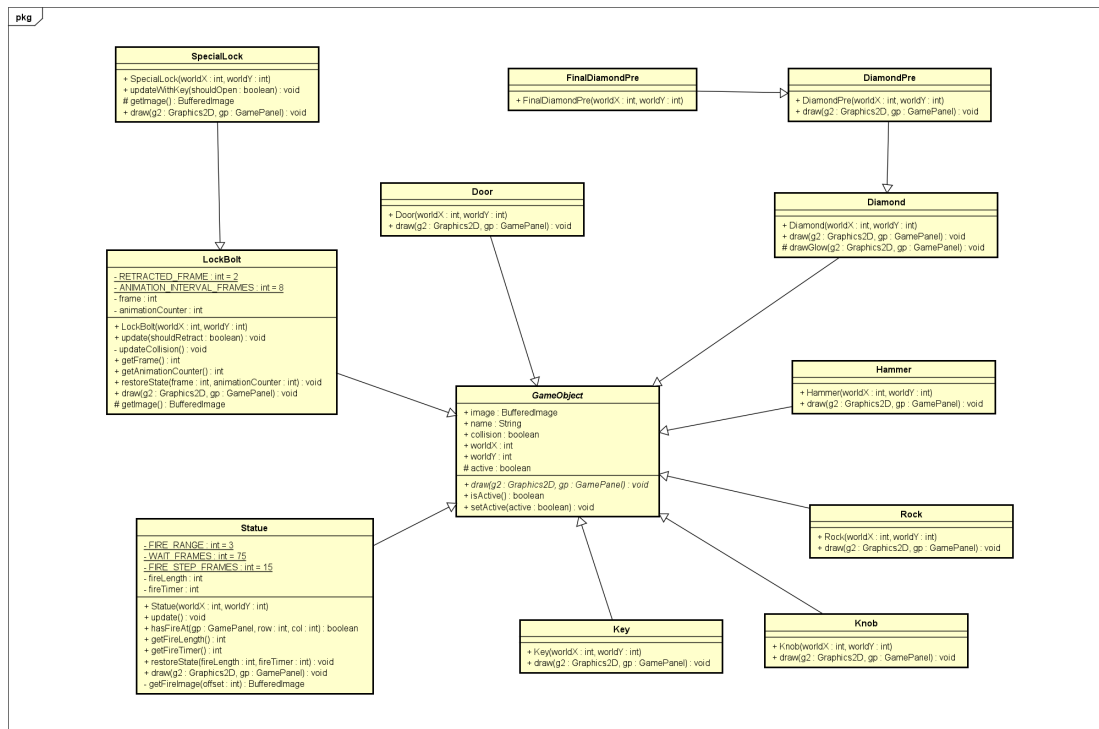
Player liên kết với KeyHandler để đọc trạng thái phím bấm và liên kết với MapLoader để kiểm tra bản đồ khi di chuyển. Điều này thể hiện luồng xử lý cơ bản: người chơi nhấn

phím, KeyHandler ghi nhận input, Player xử lý di chuyển, sau đó kiểm tra va chạm thông qua MapLoader và dữ liệu trong GamePanel.

Phần giao diện được quản lý bởi UI, GameStateManager và GameState. GameState là kiểu liệt kê các trạng thái của game như MENU, WORLD_MAP, PLAYING, PAUSED, GAME_OVER và WIN. GameStateManager dựa vào trạng thái hiện tại để cập nhật và vẽ màn hình tương ứng. Cách thiết kế này giúp tách biệt các trạng thái giao diện, tránh để toàn bộ logic menu, gameplay và màn hình kết thúc lẫn vào nhau.

3.5.3 Sơ đồ lớp nhóm object và vật phẩm

Sơ đồ lớp thứ hai tập trung vào nhóm object trong bản đồ. Đây là các đối tượng mà người chơi có thể nhìn thấy hoặc tương tác trong màn chơi, ví dụ như kim cương, đá, cửa, búa, chìa khóa, công tắc, khóa và tượng phun lửa.



Hình 3.10: Sơ đồ lớp nhóm object và vật phẩm trong game

Trung tâm của sơ đồ là lớp GameObject. Đây là lớp cha chung cho các object xuất hiện trong bản đồ. Lớp này lưu các thuộc tính chung như ảnh hiển thị, tên object, trạng thái va chạm, tọa độ trong thế giới game và trạng thái hoạt động. Các object cụ thể như Diamond, Rock, Door, Hammer, Key, Knob, LockBolt và Statue đều kế thừa từ GameObject.

Việc thiết kế GameObject làm lớp cha giúp giảm sự lặp lại mã nguồn. Thay vì mỗi object tự định nghĩa lại toàn bộ các thuộc tính như tọa độ, ảnh, trạng thái active hoặc collision, các class con có thể kế thừa những thuộc tính và hành vi chung từ GameObject. Mỗi class con chỉ cần bổ sung hoặc ghi đè những hành vi riêng của mình.

Nhóm kim cương được thiết kế theo chuỗi kế thừa gồm Diamond, DiamondPre và FinalDiamondPre. Trong đó, DiamondPre kế thừa từ Diamond, còn FinalDiamondPre kế thừa từ DiamondPre.

Cách thiết kế này phù hợp vì các loại kim cương có nhiều đặc điểm chung, nhưng có thể khác nhau về hình ảnh hoặc ý nghĩa trong gameplay.

Rock là object đại diện cho đá. Đây là vật cản có thể được người chơi đẩy và có thể tham gia vào cơ chế rơi hoặc trượt. Door là cửa để hoàn thành màn chơi. Hammer và Key là các vật phẩm hỗ trợ, giúp người chơi tương tác thêm với môi trường. Knob là công tắc, có thể dùng để điều khiển một số cơ chế trong map.

Hai lớp LockBolt và SpecialLock thể hiện quan hệ kế thừa giữa khóa thông thường và khóa đặc biệt. SpecialLock kế thừa từ LockBolt, đồng thời bổ sung hành vi mở khóa dựa trên chìa khóa của người chơi. Đây là ví dụ cho thấy kế thừa có thể được dùng để mở rộng chức năng từ một lớp đã có sẵn.

Lớp Statue đại diện cho tượng phun lửa. Khác với các object tĩnh như cửa hoặc kim cương, Statue có trạng thái riêng như độ dài lửa, thời gian chờ và khả năng kiểm tra người chơi có nằm trong vùng lửa hay không. Điều này cho thấy các object trong game không chỉ là hình ảnh tĩnh mà còn có thể mang logic xử lý riêng.

3.5.4 Nhận xét về thiết kế sơ đồ lớp

Hai sơ đồ lớp cho thấy dự án đã vận dụng được các nội dung chính của lập trình hướng đối tượng. Tính kế thừa được thể hiện rõ qua hai cây kế thừa lớn: cây Entity cho các thực thể động và cây GameObject cho các object trong map. Tính đa hình được thể hiện thông qua việc các object con cùng kế thừa từ GameObject và có thể được quản lý trong cùng một danh sách object. Tính trừu tượng được thể hiện qua các lớp tổng quát như Entity và GameObject. Tính đóng gói được thể hiện ở việc mỗi lớp phụ trách một nhóm dữ liệu và hành vi riêng.

Tuy nhiên, sơ đồ cũng cho thấy GamePanel đang là lớp có nhiều liên kết nhất. Điều này phản ánh đúng vai trò trung tâm của GamePanel, nhưng đồng thời cũng cho thấy đây là lớp có nhiều trách nhiệm. Trong hướng phát triển, nhóm có thể tách một số phần trong GamePanel thành các lớp quản lý riêng như LevelManager, ObjectManager, EnemyManager, CollisionManager hoặc SaveManager để chương trình dễ bảo trì hơn.

3.6 Đánh giá chung kết quả đạt được

Nhìn chung, dự án đã đạt được các yêu cầu cơ bản của một bài tập lớn môn Lập trình hướng đối tượng. Sản phẩm có thể chạy được, có gameplay rõ ràng, có nhiều đối tượng tương tác và có sự vận dụng các kiến thức OOP vào thiết kế chương trình.

Điểm mạnh của dự án là đã xây dựng được một game có nhiều thành phần khác nhau như bản đồ, nhân vật, object, enemy, giao diện, âm thanh và lưu tiến trình. Các sơ đồ thiết kế cho thấy mã nguồn không chỉ được viết theo hướng xử lý tuần tự, mà đã được chia thành nhiều lớp có vai trò riêng. Đặc biệt, hai lớp cha Entity và GameObject giúp thể hiện rõ tư duy thiết kế hướng đối tượng trong quá trình xây dựng game.

Bên cạnh đó, việc sử dụng MapLoader để đọc bản đồ từ file CSV giúp tách dữ liệu màn chơi khỏi mã nguồn. Cách làm này giúp việc thêm hoặc chỉnh sửa map trở nên thuận tiện hơn. Hệ thống GameStateManager và GameState cũng giúp quản lý các màn hình của game rõ ràng hơn, ví dụ như menu, gameplay, pause, game over và win.

Tuy nhiên, dự án vẫn còn một số hạn chế. Lớp `GamePanel` hiện còn đảm nhiệm nhiều trách nhiệm, bao gồm quản lý gameplay, object, enemy, camera, trạng thái game và một số logic khác. Điều này làm lớp trở nên lớn và khó bảo trì khi game tiếp tục mở rộng. Trong tương lai, nhóm có thể tách thêm các lớp quản lý riêng để hệ thống rõ ràng hơn, đúng tinh thần phân tách trách nhiệm trong lập trình hướng đối tượng.

CHƯƠNG 4. KẾT LUẬN

4.1 Hướng phát triển cho bài tập lớn

Trong tương lai, nhóm có thể tiếp tục phát triển game theo các hướng sau:

- Tách lớp GamePanel thành các lớp quản lý riêng như LevelManager, ObjectManager, EnemyManager, CollisionManager, SaveManager để giảm độ phức tạp.
- Cải thiện OOP bằng cách thêm phương thức tương tác trực tiếp trong GameObject, giảm việc sử dụng nhiều instanceof.
- Thêm nhiều màn chơi mới với độ khó tăng dần.
- Cải thiện enemy AI, ví dụ enemy có thể đuổi theo người chơi trong phạm vi nhất định.
- Thêm boss, vật phẩm hỗ trợ và chướng ngại mới.
- Hoàn thiện giao diện, hiệu ứng animation và âm thanh để game sinh động hơn.
- Bổ sung nhiều test tự động cho các logic quan trọng như va chạm, load map, save/load và điều kiện thắng thua.

4.2 Những điều học được từ bài tập lớn và môn học

Thông qua bài tập lớn này, nhóm đã học được cách áp dụng kiến thức lập trình hướng đối tượng vào một sản phẩm cụ thể. Thay vì chỉ viết các chương trình nhỏ, nhóm phải tổ chức nhiều lớp, nhiều package và nhiều thành phần liên kết với nhau để tạo thành một game hoàn chỉnh.

Về kiến thức OOP, nhóm hiểu rõ hơn vai trò của đóng gói, kế thừa, đa hình và trừu tượng. Các kiến thức này giúp chương trình dễ đọc, dễ sửa và dễ mở rộng hơn. Ví dụ, việc xây dựng các lớp cha như Entity và GameObject giúp tránh lặp lại mã nguồn giữa các đối tượng có đặc điểm chung.

Về kỹ năng lập trình, nhóm học được cách xây dựng game loop, xử lý input bàn phím, vẽ đồ họa bằng Java2D, đọc dữ liệu từ file CSV, quản lý tài nguyên ảnh và âm thanh, xử lý va chạm và quản lý trạng thái game. Đây là những kỹ năng quan trọng khi xây dựng các ứng dụng có giao diện và tương tác thời gian thực.

Về kỹ năng làm việc nhóm, nhóm nhận ra tầm quan trọng của việc phân chia công việc rõ ràng, trao đổi thường xuyên, kiểm thử sản phẩm và tổng hợp báo cáo. Bài tập lớn cũng giúp các thành viên hiểu rằng một sản phẩm phần mềm không chỉ cần chạy được, mà còn cần có cấu trúc mã nguồn hợp lý, dễ bảo trì và có tài liệu mô tả rõ ràng.

4.3 Kết luận chung

Nhìn chung, bài tập lớn đã giúp nhóm xây dựng được một sản phẩm game Diamond Rush 2D có các chức năng cơ bản và vận dụng được nhiều nội dung quan trọng của môn Lập trình hướng đối tượng. Mặc dù sản phẩm vẫn còn một số hạn chế, đặc biệt là về cấu trúc mã nguồn, giao diện và khả năng mở rộng, nhưng đây là nền tảng tốt để nhóm tiếp tục phát triển game hoàn thiện hơn trong tương lai.

PHỤ LỤC

1. Hướng dẫn chạy chương trình

Dự án đã được đóng gói sẵn thành file `DiamondRush2D.jar` trong thư mục `release`. Để chạy chương trình, máy tính cần cài đặt Java Runtime Environment hoặc JDK phiên bản 16 trở lên.

Có thể kiểm tra phiên bản Java bằng lệnh:

```
java -version
```

Kết quả cần hiển thị phiên bản Java từ 16 trở lên, ví dụ:

```
java version "17.0.x"
```

1.1. Chạy bằng file JAR

Đây là cách chạy chính của sản phẩm sau khi đã đóng gói. Người dùng chỉ cần mở Terminal, Command Prompt hoặc PowerShell tại thư mục gốc của project, sau đó chạy file JAR trong thư mục `release`.

Trên Windows PowerShell:

```
java -jar release\DiamondRush2D.jar
```

Trên Linux hoặc macOS:

```
java -jar release/DiamondRush2D.jar
```

Sau khi chạy lệnh trên, cửa sổ game Diamond Rush 2D sẽ được mở và người chơi có thể bắt đầu chơi game từ giao diện menu chính.

1.2. Chạy trực tiếp từ mã nguồn

Ngoài cách chạy bằng file JAR, có thể biên dịch và chạy trực tiếp từ mã nguồn để kiểm tra hoặc phát triển thêm chương trình.

Trên Linux hoặc macOS:

```
rm -rf out
javac -d out $(find src -name "*.java")
java -cp out:res main.Game
```

Trên Windows PowerShell:

```
Remove-Item -Recurse -Force out -ErrorAction SilentlyContinue
javac -d out (Get-ChildItem -Recurse src -Filter *.java).FullName
java -cp "out;res" main.Game
```

Trong đó, thư mục src chứa mã nguồn Java, thư mục res chứa tài nguyên của game như hình ảnh, âm thanh và dữ liệu bản đồ.

2. Cấu trúc thư mục chính của project

Cấu trúc thư mục chính của project được tổ chức như sau:

```
diamond-rush-java/
|-- src/
|   |-- main/
|   |-- config/
|   |-- core/
|   |-- entity/
|   |-- input/
|   |-- map/
|   |-- object/
|   |-- ui/
|   |-- util/
|   |-- Game.java
|
|-- res/
|   |-- characters/
|   |-- maps/
|   |-- objects/
|   |-- sounds/
|   |-- tiles/
|   |-- ui/
|
|-- diagram/
|   |-- usecase_diagram.png
|   |-- class_diagram_core.png
|   |-- class_diagram_object.png
|
|-- release/
|   |-- DiamondRush2D.jar
|
|-- README.md
```

Vai trò của các thư mục chính:

Bảng 4.1: Vai trò các thư mục chính trong project

Thư mục	Vai trò
src	Chứa toàn bộ mã nguồn Java của game.
res	Chứa tài nguyên sử dụng trong game như hình ảnh, âm thanh, tile, object và dữ liệu map.
diagram	Chứa các sơ đồ thiết kế của hệ thống, bao gồm sơ đồ use case và sơ đồ class.
release	Chứa file DiamondRush2D.jar đã đóng gói, dùng để chạy game trực tiếp.
README.md	Chứa mô tả ngắn gọn và hướng dẫn cơ bản của project.

3. Link repository

Mã nguồn của dự án được lưu trữ tại GitHub:

<https://github.com/huyyhoangg11/diamond-rush-java.git>